

UNIVERSIDAD SAN CARLOS DE GUATEMALA

FACULTAD DE CIENCIAS DE LA INGENIERÍA

INTELIGENCIA ARTIFICIAL 1

ING. LUIS FERNANDO ESPINO

AUX. JOSE ANDRES MONTENEGRO SANTOS



DOCUMENTACIÓN FASE 2

POR:

201230463 - 2253 62503 0901 - SUM COYOY, RANDY MARCELINO

202031064 - 3133 35931 0901 - OVALLE DE LEÓN, CARLOS ALEXIS

202031683 - 3353 15267 0901 - GORDILLO GONZÁLEZ , PEDRO RICARDO

GUATEMALA, GUATEMALA, DICIEMBRE DE 2024

Índice

Descripción del Proyecto: Chatbot con TensorFlow.js.....	3
Versiones Utilizadas.....	3
Descripción del Modelo.....	4
Pesos y Estructura del Modelo.....	4
Formato del Modelo.....	4
Objetivo del Modelo.....	4
Entrenamiento.....	5
Características del modelo utilizado.....	8
Traducción de las entradas.....	9
Estructura del Proyecto Frontend.....	10
Flujo Completo.....	11
Alojamiento en GitHub Pages.....	12
Versionamiento semántico.....	12

Descripción del Proyecto: Chatbot con TensorFlow.js

Descargar el proyecto del repositorio:

git clone [git@github.com:Peter2510/IA1_Proyecto_10.git](https://github.com/Peter2510/IA1_Proyecto_10.git)

Versiones Utilizadas

- **TensorFlow.js (tfjs-layers v4.22.0):**

Esta es la versión de TensorFlow.js que se utilizó para crear y entrenar el modelo de la red neuronal. TensorFlow.js es una librería de JavaScript que permite entrenar e implementar modelos de machine learning directamente en el navegador o en Node.js. La versión 4.22.0 de tfjs-layers es una versión de esta librería que facilita la creación de redes neuronales utilizando capas predefinidas como Dense y otras estructuras comunes en modelos de aprendizaje profundo.

- **Keras:**

El modelo se basó en Keras, que es una API de alto nivel para la construcción y entrenamiento de redes neuronales en Python. Aunque Keras originalmente es parte de TensorFlow, en este caso está integrado en TensorFlow.js. Esta integración facilita el uso de las capas, optimizadores y otras funcionalidades que Keras proporciona para crear redes neuronales en JavaScript. Keras es conocido por ser fácil de usar y por su diseño modular que permite construir modelos complejos de manera sencilla.

- **Backend: TensorFlow.js**

El backend utilizado para ejecutar los cálculos y entrenamientos es TensorFlow.js, lo que indica que todo el proceso de entrenamiento y predicción de la red neuronal se realiza en JavaScript. Esto permite que el chatbot pueda ser implementado tanto en el navegador web como en un entorno de Node.js sin necesidad de herramientas adicionales como Python o servidores especializados.

Descripción del Modelo

El modelo es una red neuronal con arquitectura **secuencial** creada con capas densas (Dense). Aquí se explica su estructura:

- **Capa de entrada (Input Layer):**
 - **Nombre de la capa:** dense_Dense1
 - **Unidades:** 128
 - **Activación:** ReLU (Rectified Linear Unit)
 - **Forma de entrada:** [null, 220]
Esto significa que la entrada al modelo tiene 220 características
 - **Propósito de la capa:**
Esta capa recibe la entrada procesada (el vector que representa las palabras de la pregunta) y realiza una transformación no lineal utilizando la función de activación ReLU, que es ampliamente utilizada en redes neuronales por su capacidad para manejar de manera eficiente las relaciones no lineales.
- **Capa oculta (Hidden Layer):**
 - **Nombre de la capa:** dense_Dense2
 - **Unidades:**
 - **Activación:** Softmax
 - **Propósito de la capa:**
Esta capa es responsable de tomar la salida de la capa anterior
La función de activación Softmax se utiliza para convertir los valores de salida en probabilidades, de modo que la suma de todas las probabilidades sea 1. La probabilidad más alta indica la respuesta más probable a la pregunta realizada por el usuario.

Pesos y Estructura del Modelo

- **Pesos de las capas:** Los pesos de las capas están definidos en el archivo weights.bin, que contiene los valores de los parámetros aprendidos durante el proceso de entrenamiento.

Formato del Modelo

El modelo está guardado en el formato **"layers-model"**. Este formato es comúnmente utilizado para almacenar modelos entrenados en TensorFlow.js. Contiene tanto la arquitectura de la red neuronal como los pesos entrenados, lo que permite cargar y usar el modelo en aplicaciones web o en otros entornos JavaScript.

Objetivo del Modelo

Este modelo de chatbot tiene como objetivo recibir preguntas del usuario, codificarlas como vectores y luego predecir la respuesta más apropiada de un conjunto predefinido de

respuestas. El modelo utiliza una red neuronal simple con dos capas densas para procesar la entrada y generar una salida en forma de texto.

Entrenamiento

El modelo entrenado en TensorFlow.js con Keras como API se utiliza para predecir respuestas en un chatbot. El modelo consta de una capa de entrada de 220 unidades, seguida de una capa oculta de 128 unidades con activación ReLU, y una capa de salida de 59 unidades con activación softmax para generar las respuestas posibles. Los pesos del modelo están guardados en un archivo binario que acompaña al archivo model.json, el cual describe la arquitectura y los parámetros del modelo.

Este enfoque permite que el chatbot sea implementado en un entorno de JavaScript, ya sea en un servidor con Node.js o directamente en el navegador del usuario.

El programa se divide en las siguientes secciones:

Importación de Módulos

```
const tf = require("@tensorflow/tfjs-node");
const fs = require("fs");
const readline = require("readline");
const path = require('path');
```

Se importan módulos esenciales:

- @tensorflow/tfjs-node para manipular TensorFlow.
- fs para leer y escribir archivos.
- path para trabajar con rutas del sistema.

Carga de Datos

```
const data = JSON.parse(fs.readFileSync("dialogues.json", "utf8"));
```

Carga el archivo dialogues.json que contiene datos en formato JSON. Este archivo debe tener un arreglo de objetos con un campo text. Este contiene los inputs de entrada y salida en español e inglés respectivamente.

Preprocesamiento de Datos

```
const preprocessData = (data) => {
  const pairs = [];
  for (let i = 0; i < data.length - 1; i++) {
    pairs.push({ input: data[i].text, output: data[i + 1].text });
  }
}
```

```

    }
    return pairs;
};

const pairs = preprocessData(data);

```

Normalización

```

const normalizedData = originalData.map(item => {
  if (typeof item.text === 'string') {
    item.text = item.text.normalize('NFD')
      .replace(/([aeio])\u0301|(u)\u0301\u0308/gi, "$1$2")
      .normalize();
  }
  return item;
});

```

Este paso realiza la normalización del texto, esto para limpiar el texto de caracteres especiales como tildes, para mejorar el entrenamiento y procesamiento del texto.

Creación del Vocabulario

```

const createVocabulary = (data) => {
  const vocab = new Set();
  data.forEach((text) => {
    text.split(" ").forEach((word) => vocab.add(word.toLowerCase()));
  });
  return Array.from(vocab);
};

```

```

const vocab = createVocabulary(pairs.map((p) => p.input));
const vocabSize = vocab.length;

```

- Genera un conjunto único de palabras en el conjunto de entrada.
- Convierte el conjunto en un arreglo ordenado y calcula el tamaño del vocabulario (vocabSize).

Creación de Índices para Respuestas

```

const uniqueResponses = Array.from(new Set(pairs.map((pair) => pair.output)));
const responseToIndex = uniqueResponses.reduce((obj, response, index) => {
  obj[response] = index;

```

```

    return obj;
  }, {});
const indexToResponse = Object.fromEntries(
  Object.entries(responseToIndex).map(([k, v]) => [v, k])
);

```

Crea un índice único para cada respuesta:

- responseToIndex asigna un número a cada respuesta.
- indexToResponse permite traducir un índice de vuelta a texto.

Codificación de Textos

```

const encodeText = (text, vocab) => {
  const encoded = Array(vocabSize).fill(0);
  text.toLowerCase().split(" ").forEach((word) => {
    const index = vocab.indexOf(word);
    if (index !== -1) {
      encoded[index] = 1;
    }
  });
  return encoded;
};

```

Preparación de Datos de Entrenamiento

```

const inputs = pairs.map((pair) => encodeText(pair.input, vocab));
const outputs = pairs.map((pair) => responseToIndex[pair.output]);

```

```

const outputsOneHot = tf.oneHot(tf.tensor1d(outputs, "int32"), uniqueResponses.length);

```

- inputs: Codificaciones de las entradas.
- outputs: Índices de las respuestas.
- outputsOneHot: Conversión de índices de salida en vectores one-hot.

Creación del Modelo

```

const createModel = () => {
  const model = tf.sequential();
  model.add(
    tf.layers.dense({ inputShape: [vocabSize], units: 128, activation: "relu" })
  );
  model.add(tf.layers.dense({ units: uniqueResponses.length, activation: "softmax" }));
  model.compile({
    optimizer: tf.train.adam(),

```

```

    loss: "categoricalCrossentropy",
    metrics: ["accuracy"],
  });
  return model;
};

```

```
const model = createModel();
```

Define una red neuronal con:

- Capa densa con 128 unidades y activación ReLU.
- Capa de salida con activación softmax para clasificación.

Entrenamiento del Modelo

```

const train = async () => {
  const inputTensors = tf.tensor2d(inputs);
  await model.fit(inputTensors, outputsOneHot, {
    epochs: 700,
    batchSize: 16,
  }).then(async()=>{
    await model.save('file://' + dirPath);
  });
};

```

Se utiliza un modelo de **red neuronal densa (Dense Neural Network)** implementado con TensorFlow.js. Este modelo es un tipo de red neuronal completamente conectada donde cada capa tiene conexiones con todas las neuronas de la capa anterior.

Características del modelo utilizado

1. **Modelo Secuencial:**
 - Se creó utilizando `tf.sequential()`, que permite apilar capas de manera secuencial para crear modelos simples y lineales.
2. **Capas del modelo:**
 - **Primera capa densa (Dense Layer):**
 - **Unidades:** 128 neuronas.
 - **Función de activación:** ReLU (Rectified Linear Unit).
 - **Propósito:** Capturar características importantes del vector de entrada (representación codificada del texto).
 - **Segunda capa densa (Dense Layer):**
 - **Unidades:** Igual al número de respuestas únicas.
 - **Función de activación:** Softmax.
 - **Propósito:** Generar una distribución de probabilidad sobre las posibles respuestas.

3. Tamaño de entrada:

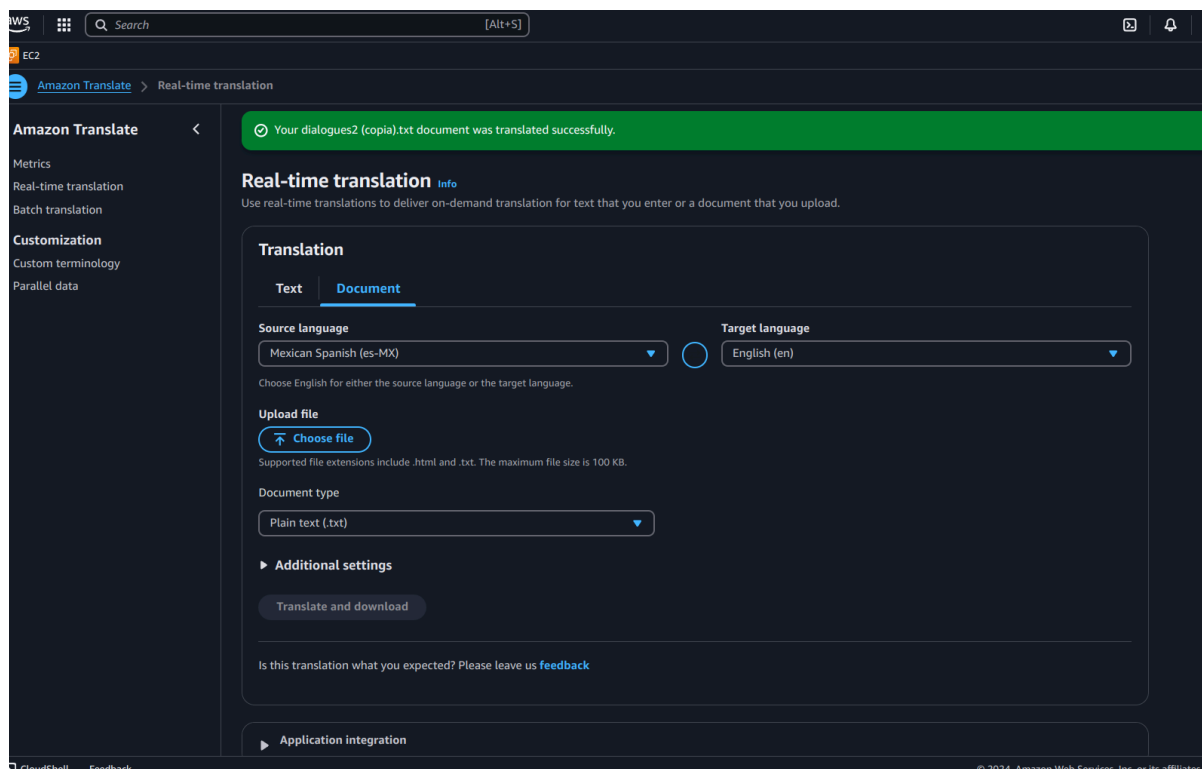
- Determinado por el tamaño del vocabulario (vocabSize), que corresponde al número total de palabras únicas en los datos de entrada.

4. Optimización y pérdida:

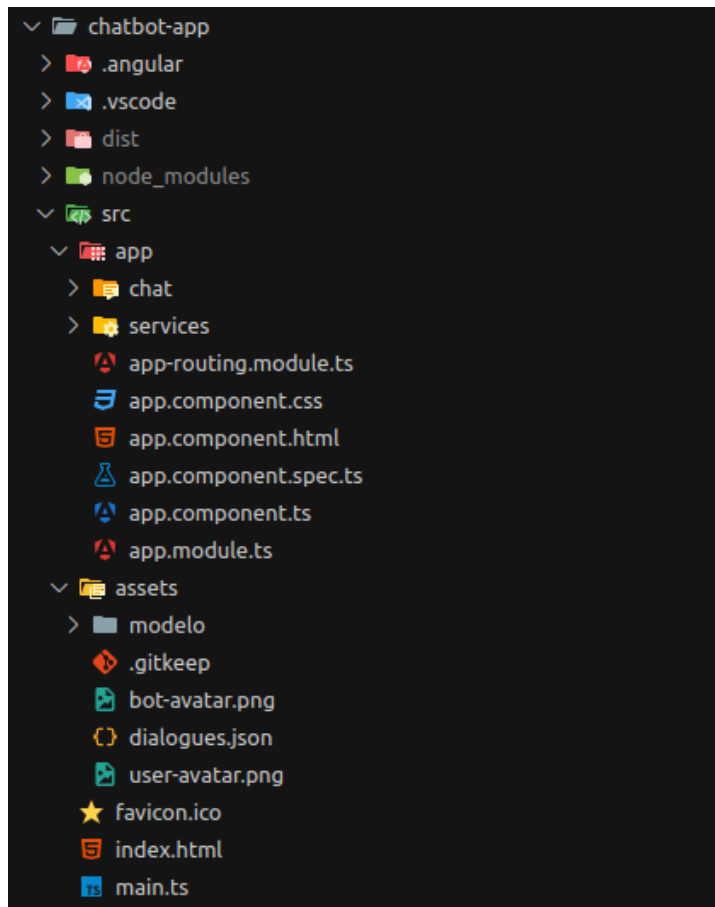
- **Optimizador:** Adam (tf.train.adam()), un algoritmo de optimización basado en gradientes que ajusta los pesos del modelo para minimizar la pérdida.
- **Función de pérdida:** categoricalCrossentropy, adecuada para problemas de clasificación multiclase con salidas codificadas como one-hot.

Traducción de las entradas

Para realizar la traducción de las entradas, se hizo uso del servicio de AWS Amazon Translate, este obtiene como entrada un archivo en español y devuelve como salida un archivo en su equivalente en inglés, esto para que sea una traducción fiable y confiable, ya que es un servicio especializado de traducción de texto.



Estructura del Proyecto Frontend



El componente **ChatComponent** gestiona la interacción entre el usuario y el chatbot. Este utiliza TensorFlow.js para procesar un modelo preentrenado y generar respuestas. Los archivos necesarios se encuentran en la carpeta `assets/model`, que contiene:

- **model.json**: Estructura del modelo neuronal.
- **weights.bin**: Pesos del modelo.
- **dialogues.json**: Datos de diálogos para entrenar y preprocesar las respuestas.

Archivo: `chat.component.ts`

El componente realiza las siguientes funciones principales:

1. **Carga del Modelo y Datos:**
 - **loadModel()**: Carga un modelo preentrenado desde `assets/modelo/model.json`.
 - **loadDialogs()**: Obtiene los datos del archivo `dialogues.json` para construir pares de entrada y respuesta.
2. **Preprocesamiento de Datos:**
 - Los diálogos se transforman en pares de entrada y salida utilizando **preprocessData()**.

- Se genera un vocabulario único con **createVocabulary()**, que almacena todas las palabras presentes en los diálogos, eliminando duplicados.
 - Las entradas se codifican en vectores binarios utilizando **encodeText()**.
3. **Generación de Respuestas:**
- **predictionsModel(input)** procesa el texto de entrada, lo transforma en un tensor y utiliza el modelo para predecir la respuesta más adecuada.
4. **Gestión de Mensajes:**
- **sendMessage()** maneja los mensajes enviados por el usuario, genera la respuesta del chatbot y actualiza la lista de mensajes para reflejarla en la interfaz.

Métodos Clave

ngOnInit()

Se ejecuta al inicializar el componente:

- Llama a **loadModel()** para cargar el modelo.
- Llama a **loadDialogs()** para cargar los datos de diálogos.
- Realiza el preprocesamiento y prepara los datos necesarios para las predicciones.

Obtiene los diálogos en formato JSON desde la carpeta de activos y los almacena para el preprocesamiento.

sendMessage()

Gestiona la interacción entre el usuario y el chatbot:

- Agrega el mensaje del usuario a la lista.
- Genera la respuesta del chatbot llamando a **predictionsModel()**.
- Añade la respuesta generada al historial de mensajes después de un breve retraso.

Flujo Completo

1. **Carga Inicial:**
 - Se cargan el modelo y los datos de diálogos desde los archivos en `assets/model/`.
 - Los datos se pre procesan para generar pares de entrada y salida, un vocabulario y representaciones codificadas.
2. **Interacción:**
 - El usuario ingresa texto en la interfaz.
 - **sendMessage()** gestiona el mensaje y llama al modelo para obtener una respuesta.
 - La interfaz se actualiza con el mensaje del usuario y la respuesta del chatbot.
3. **Predicción:**
 - El texto de entrada se codifica con el vocabulario.

- Se pasa al modelo neuronal, que devuelve un índice correspondiente a la respuesta generada.
- Se traduce el índice a texto legible utilizando el diccionario `indexToResponse`.

Consideraciones Técnicas

1. **Estructura del vocabulario:** El vocabulario generado depende de los datos en `dialogues.json`. Actualizar este archivo implica regenerar el vocabulario.
2. **Codificación de Entradas:** Utiliza una representación binaria basada en la presencia o ausencia de palabras. Esto limita el modelo a predecir en base al vocabulario disponible.
3. **Conexión Asíncrona:** Tanto la carga del modelo como de los diálogos usan promesas, asegurando que el componente no procese predicciones hasta que los recursos estén completamente cargados.

Alojamiento en GitHub Pages

Para construir el proyecto

```
ng build --configuration production --base-href  
"https://Peter2510.github.io/IA1_Proyecto_10/"
```

Para desplegar el proyecto

```
npx angular-cli-ghpages --dir=dist/chatbot-app
```

Versionamiento semántico

Angular

Se hizo uso de angular 16.2 para el desarrollo del frontend

Node

Para realizar la generación del modelo se hizo uso de la versión 18